

Technical Report: Laptop Price Prediction Using FCNN

Project Phase: Part-3

Target Platform: PC

1. Introduction

1.1 Project Overview

This project addresses a supervised regression problem focused on predicting laptop prices using a fully connected neural network (FCNN). The work is carried out entirely on a personal computer (PC) and emphasizes proper data preprocessing, feature extraction, model design, training, and evaluation using real-world data.

1.2 Objectives

The objectives of this project are:

- To formulate laptop price estimation as a regression problem.
- To preprocess and encode mixed numerical and categorical features.
- To design and train a fully connected neural network for price prediction.
- To evaluate the model using standard regression performance metrics.

2. Problem Definition

2.1 Problem Statement

The task is to predict the market price of a laptop in euros based on its technical specifications and configuration details. Laptop prices are influenced by multiple nonlinear factors such as hardware components, brand, and display characteristics, making this a suitable application for neural network-based regression.

2.2 Input and Output Requirements

- **Input:**
Laptop specifications including screen size, RAM, CPU frequency, weight, screen resolution, manufacturer, laptop type, storage configuration, GPU, and operating system.

- **Output:**
A single continuous value representing the predicted laptop price in euros.

2.3 Performance Constraints

The primary performance objective is to achieve accurate and generalizable predictions. Model quality is evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2).

3. Dataset Description

3.1 Dataset Source

The dataset used is the **Laptop Price Dataset**, a publicly available dataset containing real laptop specifications and corresponding prices. It is widely used for regression and price estimation studies.

3.2 Data Format and Features

Each row in the dataset represents a single laptop. The dataset contains:

- **Numerical features:** Screen size (inches), RAM (GB), CPU frequency (GHz), weight (kg), screen width, and screen height.
- **Categorical features:** Manufacturer, laptop type, memory configuration, GPU model, and operating system.
- **Target variable:** Laptop price in euros.

3.3 Preprocessing

The preprocessing steps included:

- Converting RAM and weight from string format to numeric values.
- Extracting CPU frequency from textual descriptions.
- Parsing screen resolution into width and height.
- One-hot encoding categorical variables.
- Standardizing numerical features using z-score normalization.

These steps ensured that all features were suitable for neural network training.

```

# Feature Extraction and Cleaning
# 1. Convert RAM and Weight to numeric
df["Ram"] = df["Ram"].str.replace("GB", "").astype(int)
df["Weight"] = df["Weight"].str.replace("kg", "").astype(float)

# 2. Extract CPU Frequency (GHz) via Regex
df["Cpu_GHz"] = df["Cpu"].str.extract(r'(\d+\.\d+|\d+)GHz').astype(float)

# 3. Parse Screen Resolution
res = df["ScreenResolution"].str.extract(r'(\d+)x(\d+)')
df["ScreenWidth"] = res[0].astype(int)
df["ScreenHeight"] = res[1].astype(int)

# 4. Define Transformations Pipeline
numeric_transformer = StandardScaler()
categorical_transformer = OneHotEncoder(handle_unknown="ignore", sparse_output=False)

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, NUM_FEATURES),
        ("cat", categorical_transformer, CAT_FEATURES)
    ]
)

```

3.4 Train / Validation / Test Split

The dataset was split into:

- **70% training data**
- **15% validation data**
- **15% test data**

This split allowed proper model training, hyperparameter tuning, and unbiased evaluation.

```

# Splitting Data: 70% Train, 15% Validation, 15% Test
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.3, random_state=42
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42
)

# Apply preprocessing (fit on train, transform val/test)
X_train_prep = preprocessor.fit_transform(X_train)
X_val_prep = preprocessor.transform(X_val)
X_test_prep = preprocessor.transform(X_test)

```

4. Model Development on PC

4.1 Model Architecture

A fully connected neural network was designed with:

- An input layer corresponding to the preprocessed feature vector
- Three hidden layers with ReLU activation functions
- Dropout layers to reduce overfitting
- A single linear output neuron for regression

```
# FCNN Architecture Design
model = Sequential([
    # Input Layer automatically inferred from X_train_prep shape
    Dense(256, activation="relu", input_shape=(X_train_prep.shape[1],)),
    Dropout(0.3), # Dropout to prevent overfitting

    Dense(128, activation="relu"),
    Dropout(0.2),

    Dense(64, activation="relu"),

    # Single linear neuron for regression output
    Dense(1)
])
```

4.2 Training Environment

The model was developed and trained using:

- Python
- TensorFlow / Keras
- Scikit-learn
- Pandas and NumPy

All experiments were conducted on a PC-based environment.

4.3 Training Procedure

The training procedure used:

- Mean Squared Error (MSE) as the loss function
- Adam optimizer with a learning rate of 0.001
- Early stopping based on validation loss
- A maximum of 200 epochs with a batch size of 32

```

# Compile Model
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="mse",
    metrics=["mae"]
)

# Configure Early Stopping
early_stop = EarlyStopping(
    monitor="val_loss",
    patience=15,
    restore_best_weights=True
)

# Start Training
history = model.fit(
    X_train_prep, y_train,
    validation_data=(X_val_prep, y_val),
    epochs=200,
    batch_size=32,
    callbacks=[early_stop],
    verbose=1
)

```

4.4 Model Performance

The final trained model achieved the following performance on the test dataset:

- **MAE:** 180 €
- **RMSE:** 268 €
- **R²:** 0.816

These results demonstrate strong predictive accuracy and effective learning of nonlinear relationships between laptop specifications and price.

```

# Generate Predictions
y_pred = model.predict(X_test_prep).flatten()

# Calculate Performance Metrics
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print(f"MAE : {mae:.2f} €")
print(f"RMSE : {rmse:.2f} €")
print(f"R² : {r2:.3f}")

```

5. Results and Analysis

5.1 Prediction Behavior

The model produced predictions that closely followed actual laptop prices, particularly for mid-range and high-end devices. Larger errors were observed for low-priced laptops, which typically exhibit higher variability.

5.2 Performance Interpretation

An R^2 score of 0.816 indicates that over 80% of the variance in laptop prices is explained by the model. This confirms that the selected features and FCNN architecture are well-suited for the task.

6. Conclusion

6.1 Summary

This project successfully demonstrated the application of a fully connected neural network for laptop price prediction. Through careful data preprocessing, feature engineering, and model design, strong regression performance was achieved on a real-world dataset.